IMU_TO_DXL v1.2 开发手册

飞特（Feetech）舵机总线 IMU 模块 · 客户开发与测试指南

项目	参数
型号	IMU_TO_DXL v1.2
主控	CH32V203（RISC-V 96 MHz）
传感器	LSM6DSV16X（6 轴，内置 SFLP 硬件融合引擎）
协议	飞特 SCS 系列 / Dynamixel（二选一固件）
从站 ID	**200**（可改）
波特率	**1 Mbps**
物理层	单线半双工 UART，TTL 3.3 V
姿态输出	四元数 + 陀螺仪角速度
输出频率	**120 Hz**
姿态延迟	约 8.3 ms（最坏）
板子尺寸	34.0 × 42.3 mm，板厚 1.6 mm
固件升级	总线 OTA（不用拆机）

---

1. 数据块总览

IMU 作为总线上的一个从站，主机用**同步读（Sync Read）**一次把它和其他舵机一起读回来。

协议	起始地址	长度	说明
飞特 FT	**56**（0x38）	**15**	紧凑块，推荐
Dynamixel XL	124（0x7C）	12	兼容官方布局

本手册以飞特 15 字节紧凑块为主。

15 字节布局（小端）

偏移	长度	字段	类型	说明
0–1	2	gyro_x	int16	陀螺仪 X 角速度原始值
2–3	2	gyro_y	int16	陀螺仪 Y
4–5	2	gyro_z	int16	陀螺仪 Z
6–7	2	quat_x	IEEE754 half	四元数 X
8–9	2	quat_y	IEEE754 half	四元数 Y
10–11	2	quat_z	IEEE754 half	四元数 Z
12	1	sequence	uint8	**只在有新四元数时 +1**
13	1	status	uint8	状态位，见第 3 节
14	1	reserved	uint8	保留，固定 0

---

2. 读数据：完整帧格式

2.1 主机发出的同步读请求

```
FF FF  FE  LEN  82  ADDR  RLEN  ID1  ID2  ...  IDn  CHK
```

字段	值	说明
`FF FF`	—	帧头
`FE`	—	广播地址
`LEN`	请求的总长度	从 ID 字段算到 CHK
`82`	—	SYNC_READ 指令
`ADDR`	**0x38**	起始地址 56
`RLEN`	**0x0F**	读 15 字节
`ID1`	**0xC8**	**本模块 ID 200，必须排在第一个**
`ID2..IDn`	—	其他舵机 ID
`CHK`	—	`~(ID + LEN + INST + Σparams) & 0xFF`

实测示例（16 个设备，200 在首位）：

```
FF FF FE 14 82 38 0F C8 14 15 16 17 18 1E 1F 20 21 0A 0B 0C 0D 0E XX
```

这里 `LEN = 0x14 = 20`、`ID1 = 0xC8 = 200`、`ADDR = 0x38 = 56`、`RLEN = 0x0F = 15`。

2.2 本模块的应答（21 字节）

```
FF FF  C8  11  00  D0 D1 ... D14  CHK
```

字段	值	说明
`FF FF`	—	帧头
`C8`	200	本模块 ID
`11`	17	`LEN = 数据长度 15 + 2`
`00`	0	错误码（正常为 0）
`D0..D14`	—	上面第 1 节的 15 字节数据块
`CHK`	—	同上算法

---

3. 状态位（字节 13）详解

BIT = 1 表示异常。

位	名称	含义	建议处理
0x01	NOT_READY	还没输出过有效四元数	上电后等它清零；一直为 1 说明融合未启动
0x02	**STALE**	**距上次新四元数 > 30 ms**	**数据过期，不要用于闭环**
0x04	SELF_TEST	WHO_AM_I 校验失败	硬件异常，联系售后
0x08	**READER_SLOW**	**上次读取后又积累了 ≥5 个新样本**	**主机读得太慢，考虑提高读取频率**

推荐的主机判断逻辑

```c
if (status & 0x01) { /* 未就绪：跳过本帧 */ }
if (status & 0x02) { /* 数据过期：不要用于控制 */ }
if (status & 0x08) { /* 自己读得太慢：应提高频率或减少其他总线负载 */ }
```

用 sequence 判断"是不是新数据"

```c
static uint8_t last_seq;
if (seq != last_seq) {
    last_seq = seq;
    /* 这一帧确实是新姿态 */
}
```

`sequence` **只在拿到新四元数时 +1**，120 Hz 下每秒加 120。用它判断比看时间戳更可靠。

---

4. 总线时序要求（重要）

IMU 是总线上的**第一个从站**，所以它的应答时机最紧张——紧跟在主机发送结束之后。

```
主机指令结束
   ↓ 60 µs（协议规定的包间隔）
本模块应答开始（210 µs 发完 21 字节）
   ↓ 下一个设备在自己的时隙应答
```

项目	数值
包间隔	**60 µs**
单设备时隙	**270 µs** = 60 包间隔 + 210 应答长度
1 Mbps 下每字节	10 µs
21 字节应答	210 µs

三条必须遵守的规则：

1. **ID 200 必须排在同步读 ID 列表的第一位。** 协议按 ID 顺序分配时隙，它排第一就意味着"主机发完立刻回"。
2. **不要让其他设备插在 200 前面。** 否则它会占用别人的时隙，两边都可能出错。
3. **波特率必须精确 1 Mbps**，误差要小于 ±2%（用 12 MHz 晶振或更高的 PLL 时钟，不要用内部 RC）。

5. 快速测试（PC + USB 串口）

5.1 硬件接法

模块是**单线半双工**，PC 上需要：

- **USB 转单线半双工模块**（TTL 3.3 V），或
- USB-TTL + 一个二极管/MOS 单线转换电路，或
- 直接挂到舵机总线上，用舵机电源供电

模块引脚	接到
BAT+	电池正极（**待确认电压范围**）
GND	电池负极
BUS	单线总线（TX/RX 合并）

5.2 Python 测试脚本

```python
#!/usr/bin/env python3
"""IMU_TO_DXL v1.2 读取示例 —— 需要 USB 转单线半双工适配器"""
import serial, struct, math, time

ID_IMU   = 200
ADDR     = 56
LEN      = 15
BAUD     = 1000000

def ft_checksum(body: bytes) -> int:
    return (~sum(body)) & 0xFF

def build_sync_read(ids):
    """构造同步读请求：FF FF FE LEN 82 ADDR RLEN ID... CHK"""
    body = bytes([0xFE, len(ids) + 4, 0x82, ADDR, LEN] + list(ids))
    return b'\xFF\xFF' + body + bytes([ft_checksum(body)])

def half_to_float(h):
    s = (h >> 15) & 1
    e = (h >> 10) & 0x1F
    m = h & 0x3FF
    if e == 0:    v = m / 1024.0 * 2 ** -14
    elif e == 31: v = 0.0
    else:         v = (1 + m / 1024.0) * 2.0 ** (e - 15)
    return -v if s else v

def read_imu(port):
    req = build_sync_read([ID_IMU])
    port.reset_input_buffer()
    port.write(req)

    # 期望 21 字节：FF FF C8 11 00 <15 字节数据> CHK
    buf = port.read(21)
    if len(buf) != 21 or buf[0] != 0xFF or buf[1] != 0xFF or buf[2] != ID_IMU:
        return None
    d = buf[5:20]
    gx, gy, gz = (struct.unpack('<h', d[0:2])[0],
                  struct.unpack('<h', d[2:4])[0],
                  struct.unpack('<h', d[4:6])[0])
    qx = half_to_float(struct.unpack('<H', d[6:8])[0])
    qy = half_to_float(struct.unpack('<H', d[8:10])[0])
    qz = half_to_float(struct.unpack('<H', d[10:12])[0])
    w2 = 1.0 - qx*qx - qy*qy - qz*qz
    qw = math.sqrt(w2) if w2 > 0 else 0.0
    return {
        'gyro_dps': (gx * 0.0175, gy * 0.0175, gz * 0.0175),
        'quat':     (qw, qx, qy, qz),
        'seq':      d[12],
        'status':   d[13],
    }
```

```
if __name__ == '__main__':
    ser = serial.Serial('/dev/ttyUSB0', BAUD, timeout=0.05)
    last_seq = None
    t0 = time.time(); n = 0
    while time.time() - t0 < 5:
        r = read_imu(ser)
        if r is None:
            print('无应答'); continue
        if r['seq'] != last_seq:
            last_seq = r['seq']; n += 1
            w, x, y, z = r['quat']
            # 由四元数算倾角（机体 Z 轴与重力的夹角）
            gz = 1 - 2 * (x*x + y*y)
            tilt = math.degrees(math.acos(max(-1, min(1, -gz))))
            print(f"seq={r['seq']:3d} status=0x{r['status']:02X} "
                  f"q=({w:+.3f},{x:+.3f},{y:+.3f},{z:+.3f}) 倾角={tilt:5.1f}° "
                  f"gyro={r['gyro_dps'][0]:+6.2f},{r['gyro_dps'][1]:+6.2f},
{r['gyro_dps'][2]:+6.2f} dps")
    print(f"\n5 秒内收到 {n} 个新样本，约 {n/5:.0f} Hz（标称 120 Hz）")
```

预期输出：

```
seq= 12 status=0x00 q=(+0.999,+0.021,-0.008,+0.003) 倾角=  1.2° gyro= +0.31, -0.55,
+0.12 dps
...
5 秒内收到 598 个新样本，约 120 Hz（标称 120 Hz）
```

5.3 验收检查表

检查项	期望	不符合时看
有应答	21 字节 `FF FF C8 11 00 ...`	5 号线接法/波特率/ID
采样率	110–125 Hz	主机读得太慢（`READER_SLOW`）
静止倾角	≈ 0°（±3° 内）	安装面不平，重新标定
静止 gyro	±1 dps 内	正常零偏，融合会自动校正
`status` bit0	上电几秒后为 0	融合未启动
`status` bit1	一直为 0	主机读得太慢（>30 ms 一次）
翻转模块	倾角跟着变化	姿态解算异常

6. 常见问题

Q：读不到应答怎么办？ 按顺序排查：

1. 波特率是否精确 1 Mbps
2. 同步读里 **ID 200 是否排在第一位**
3. 请求里 `ADDR` 是否 0x38、`RLEN` 是否 0x0F
4. 单线半双工方向转换是否正常（TX 发完要及时释放总线）
5. 共地是否可靠

Q：能读到数据但倾角不对？

- 四元数需要用安装标定（`imu_mount_quat`）旋转到机体系
- 只发 3 个分量，主机必须自己补 $W = \sqrt{(1 - x^2 - y^2 - z^2)}$
- 检查 half 浮点解码是否按 16 位小端

Q：`status` 一直是 0x08（READER_SLOW）？ 主机读取间隔太长。120 Hz 下 30 ms 会积累 3.6 个新样本，5 个以上就置位。建议读取频率 ≥ 50 Hz。

Q：`status` 出现 0x02（STALE）？ 说明距上次新四元数超过 30 ms。这种情况通常意味着模块内部融合卡住或总线通信中断，检查供电和总线。

Q：陀螺仪有 1–2 dps 的零偏，正常吗？ 正常。这是消费级 IMU 的固有零偏，SFLP 会在线估计并补偿，不影响姿态输出。

Q：改动 ID 后读不到了？ 同步读列表里必须用新 ID，且仍排在第一位。
